

ЛАБОРАТОРНАЯ РАБОТА

Компонент анализа файловой системы в Docker

Node.js · node:fs · bind mount · автоматическая проверка

Ключевая идея

Docker-контейнер рассматривается как компонент более высокого уровня. Его внутренняя реализация скрыта; проверяющей системе известны только входной контракт (/data) и выходной контракт (ISU-FILES-DIRECTORIES-SIZE).

Цель работы

Разработать программный компонент на Node.js, использующий встроенный модуль файловой системы node:fs, упаковать его в Docker-образ и обеспечить однозначно проверяемый внешний контракт.

После выполнения вы должны уметь

- работать с файловой системой средствами Node.js;
- выполнять рекурсивный обход каталогов;
- выделять входной и выходной интерфейсы программного компонента;
- скрывать детали реализации за границей компонента;
- создавать Docker-образ приложения и запускать контейнер с bind mount;
- проверять решение как черный ящик по заранее заданному контракту.

1. Задание

Разработайте приложение на Node.js, которое анализирует содержимое каталога /data. Каталог /data передается приложению извне при запуске Docker-контейнера.

Приложение должно рекурсивно определить:

1. количество обычных файлов;
2. количество каталогов;
3. суммарный размер всех обычных файлов в байтах.

2. Формат результата

После завершения анализа приложение должно вывести в стандартный поток вывода stdout ровно одну строку:

```
ISU-FILES-DIRECTORIES-SIZE
```

где:

- ISU — ваш шестизначный табельный номер в ISU;
- FILES — количество обычных файлов;
- DIRECTORIES — количество каталогов;
- SIZE — суммарный размер обычных файлов в байтах.

Пример: если ISU студента — 307482, а в каталоге найдено 7 файлов, 3 каталога и 18432 байта данных, корректный вывод:

```
307482-7-3-18432
```

Важно

Никакой другой текст в stdout выводиться не должен. Не добавляйте пояснения, приветствия, диагностические сообщения или подписи. Если вам нужна отладочная информация, выводите ее в stderr либо удалите перед сдачей.

Формат результата должен соответствовать регулярному выражению:

```
^\d{6}-\d+--\d+--\d+\s*$
```

3. Правила подсчета

- Обход каталога выполняется рекурсивно.
- Исходный каталог /data в количество каталогов не включается.
- Пустые вложенные каталоги учитываются как каталоги.
- В число файлов входят только обычные файлы.
- SIZE равен сумме размеров обычных файлов в байтах.
- Символические ссылки при подсчете учитывать не требуется.
- Каталог /data используется только для чтения.

Пример подсчета

```

/data
├── a.txt    100 байт
├── b.txt    500 байт
├── docs/
│   ├── c.txt  200 байт
│   └── old/
│       └── d.txt 50 байт
└── empty/

```

Правильный результат для студента с ISU 307482:

```
307482-4-3-850
```

4. Требования к реализации

Приложение должно быть реализовано на Node.js. Для работы с файловой системой необходимо использовать встроенные средства Node.js, например:

```
import { readdir, stat } from 'node:fs';
```

Допускается использование других функций node:fs или node:fs/promises. Сторонние библиотеки для рекурсивного обхода файловой системы не требуются.

Способ внутренней организации кода выбирается самостоятельно. Проверка не должна зависеть от имен ваших функций, количества файлов исходного кода или внутренней структуры проекта.

5. Требования к Docker-образу

В корне проекта должен находиться Dockerfile. Образ должен успешно собираться командой:

```
docker build -t student-fs .
```

После сборки приложение должно запускаться без интерактивного ввода. Для самопроверки можно смонтировать тестовый каталог:

```
docker run --rm \
  --network none \
  --mount type=bind,source="$(pwd)/test-data",target=/data,readonly \
  student-fs
```

Контейнер не должен требовать:

- дополнительных аргументов командной строки;
- доступа к сети;
- изменения файлов внутри /data;
- предварительной настройки переменных окружения для выполнения основного задания.

6. Граничные случаи

- пустой каталог;
- только файлы;

- только пустые подкаталоги;
- несколько уровней вложенности;
- имена файлов с пробелами;
- файлы нулевого размера.

Для пустого каталога студент с ISU 307482 должен получить:

307482-0-0-0

7. Контракт компонента

Docker-контейнер как программный компонент		
Вход	/data	bind mount, readonly
Граница	Docker-контейнер	детали реализации скрыты
Реализация	Node.js + node:fs	внутри контейнера
Выход	ISU-FILES-DIRECTORIES-SIZE	stdout, одна строка

Почему это именно компонентная задача

В терминах первой лекции здесь одновременно видны два уровня компонентности. Модуль node:fs предоставляет готовый программный интерфейс внутри реализации, а Docker-контейнер образует внешнюю границу, через которую проверятор взаимодействует с решением как с черным ящиком.

Элемент	Роль в компонентной модели	Что знает checker
node:fs	готовый интерфейс работы с файловой системой	не обязан знать
Docker-контейнер	граница и инкапсуляция решения	образ можно запустить
/data	входной интерфейс компонента	куда монтировать данные
stdout	выходной интерфейс компонента	формат ISU-FILES-DIRECTORIES-SIZE

Практический смысл: checker может заменить способ реализации, не меняя правила взаимодействия, пока вход и выход остаются совместимыми с контрактом.

8. Краткая инструкция: основные шаги создания Docker-образа

Эта страница — техническая памятка

Ниже показан минимальный маршрут от файла программы до запуска контейнера. Код анализа файловой системы здесь намеренно не приводится полностью: его необходимо реализовать самостоятельно.

Шаг 1. Подготовьте каталог проекта

```
lab/
├── Dockerfile
└── index.mjs
```

В `index.mjs` разместите программу Node.js. Она должна читать именно `/data` и завершаться выводом одной строки заданного формата.

Шаг 2. Создайте минимальный Dockerfile

```
FROM node:22-alpine
WORKDIR /app
COPY index.mjs .
CMD ["node", "index.mjs"]
```

Этот Dockerfile берет официальный образ Node.js, копирует программу в контейнер и задает команду запуска.

Шаг 3. Соберите образ

```
docker build -t student-fs .
```

Точка в конце команды означает: использовать текущий каталог как контекст сборки.

Шаг 4. Подготовьте тестовые данные

```
mkdir -p test-data/docs test-data/empty
printf 'abc' > test-data/a.txt
printf '12345' > test-data/docs/b.txt
```

Шаг 5. Запустите контейнер с bind mount

```
docker run --rm \
  --network none \
  --mount type=bind,source="$(pwd)/test-data",target=/data,readonly \
  student-fs
```

Каталог `test-data` на вашем компьютере становится `/data` внутри контейнера. Модификатор `readonly` запрещает контейнеру изменять входные файлы.

Шаг 6. Проверьте внешний контракт

Убедитесь, что `stdout` содержит только одну строку вида `ISU-FILES-DIRECTORIES-SIZE`. Затем измените структуру `test-data` и повторите запуск. Хорошее решение не должно зависеть от заранее известных имен и количества файлов.

9. Автоматическая проверка

При проверке будут созданы неизвестные заранее тестовые каталоги, проверятор смонтирует каждый из них как /data и сравнит фактический вывод контейнера с ожидаемым результатом.

```
docker run --rm \
  --network none \
  --mount type=bind,source="/tmp/test-data",target=/data,readonly \
  student-fs
```

Первая часть ответа будет интерпретироваться как ISU. Проверять сопоставит ее со словарем группы. Если шестизначный номер отсутствует в словаре, результат может считаться неподписанным и не засчитываться.

Что проверить на этом этапе

- ☐ Dockerfile лежит в корне проекта.
- ☐ Команда `docker build -t student-fs` завершается без ошибки.
- ☐ Контейнер работает без доступа к сети и без интерактивного ввода.
- ☐ Анализируется /data, а не каталог исходного кода.
- ☐ Обход рекурсивный, корневой /data не учитывается как каталог.
- ☐ Суммарный размер считается в байтах.
- ☐ В stdout ровно одна строка.
- ☐ В строке указан ваш реальный шестизначный ISU.
- ☐ Результат соответствует шаблону ISU-FILES-DIRECTORIES-SIZE.